

Google ADK

실무 가이드 · 지식 점검

Agent Development Kit — 핵심 개념 · 패턴 · 코드 · 함정

LangGraph 경쟁 솔루션 · 에이전트 트리 + 워크플로 에이전트 아키텍처
Google ADK 기준 (2026년 6월) · 대상: 풀스택 / MLOps / DevOps 엔지니어
난이도: 실무 핵심 · 본문 한국어 + 코드/용어 영문

1. Google ADK란 무엇이고 언제 쓰는가
2. 핵심 멘탈 모델: 에이전트 트리
3. LlmAgent — 기본 에이전트
4. Workflow Agents — Sequential·Parallel·Loop
5. 멀티 에이전트 — sub_agents·transfer·AgentTool
6. Tools — 함수·내장·서드파티
7. Runner와 실행
8. Session·State·Memory
9. Callbacks — 가드레일과 개입
10. 개발 UI와 평가(adk web·adk eval)
11. 배포 — Agent Engine·Cloud Run
12. A2A — 에이전트 간 표준 프로토콜
13. ADK vs LangGraph 비교
14. 자주 하는 실수 모음
15. API 빠른 레퍼런스

지식 점검 퀴즈 (객관식 20문항)

정답 및 해설

1 Google ADK란 무엇이고 언제 쓰는가

Google ADK(Agent Development Kit)는 구글의 오픈소스 코드 우선(code-first) 멀티 에이전트 프레임워크다(Python/Java). Gemini와 구글 생태계에 최적화돼 있지만 LiteLLM 연동으로 모델 중립이며, 에이전트 트리(계층)와 워크플로 에이전트(Sequential/Parallel/Loop)로 결정적 파이프라인과 LLM 동적 라우팅을 모두 지원한다. 개발 UI(adk web), 평가(adk eval), 관리형 배포(Vertex AI Agent Engine)까지 수명주기 전체를 갖춘 것이 강점이다.

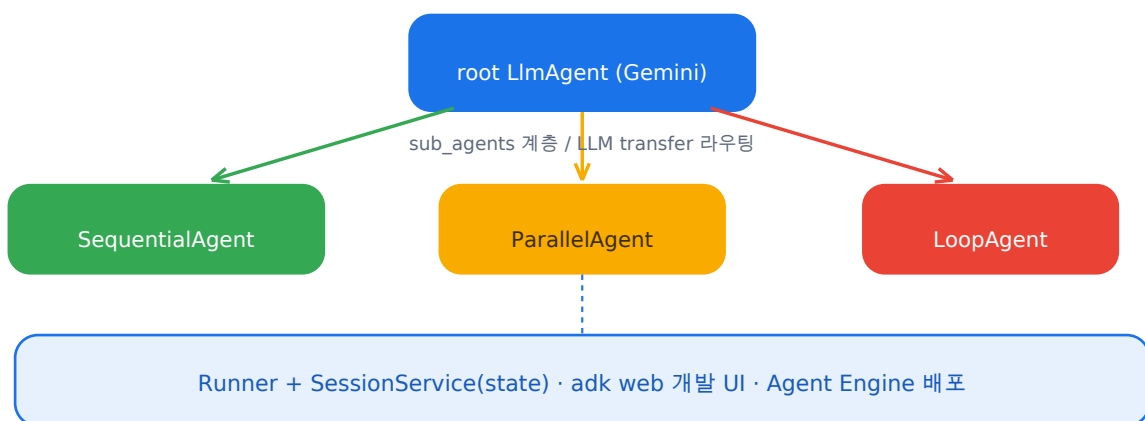
상황	권장
Gemini/GCP 중심 스택, 관리형 운영까지	ADK + Agent Engine
고정 순서 파이프라인 + 부분 병렬/반복	ADK 워크플로 에이전트
전문 에이전트 계층에 동적 위임	ADK sub_agents + transfer
타사 에이전트와 표준 통신	ADK + A2A 프로토콜
임의 그래프 토폴로지·체크포인트 정밀 제어	LangGraph 고려

한 줄 정의

ADK = "에이전트 트리 + 워크플로 에이전트로 오케스트레이션하고, Runner·Session·개발 UI·Agent Engine으로 수명주기를 받쳐주는 구글의 에이전트 프레임워크".

2 핵심 멘탈 모델: 에이전트 트리

ADK 애플리케이션은 루트 에이전트를 정점으로 한 트리다. 부모의 sub_agents로 계층을 만들고, 라우팅은 두 방식 — (1) LLM 주도 transfer: 루트 LLM이 각 하위 에이전트의 description을 보고 동적으로 위임, (2) 워크플로 에이전트: Sequential/Parallel/Loop가 결정적으로 하위 실행을 통제. 둘을 자유롭게 중첩한다.



▲ 에이전트 트리: 루트 LlmAgent 아래 워크플로 에이전트(결정적)와 LLM transfer(동적)를 중첩. Runner와 SessionService가 실행·상태를 받친다.

- **LlmAgent** — LLM이 추론·도구 사용·위임을 결정하는 기본 에이전트.
- **Workflow Agents** — LLM 없이 실행 흐름만 통제(Sequential/Parallel/Loop).
- **Runner** — 실행 엔진. **Session/State** — 대화·상태 관리. **Events** — 실행 중 발생하는 이벤트 스트림.

3 LlmAgent — 기본 에이전트

Agent(= LlmAgent)는 name·model·instruction·description·tools로 정의한다. description은 다른 에이전트가 위임 판단에 쓰는 요약이므로 명확하게. output_key를 주면 최종 응답이 세션 state에 자동 저장되어 다음 에이전트가 활용한다.

```
from google.adk.agents import Agent
from google.adk.tools import google_search

researcher = Agent(
    name="researcher",
    model="gemini-2.5-flash",
    description="Finds up-to-date facts on a topic.", # used for routing
    instruction="Research {topic} and list key facts.", # {topic} from state
    tools=[google_search],
    output_key="research_result", # save final text into session state
)
```

instruction의 상태 템플릿

instruction 문자열의 {key}는 세션 state 값으로 치환된다 — 에이전트 간 데이터 전달의 기본 통로가 output_key 저장 + {key} 주입 조합이다.

4 Workflow Agents — Sequential·Parallel·Loop

에이전트	동작	적합
SequentialAgent	sub_agents를 선언 순서대로 실행	고정 파이프라인(조사→작성→검수)
ParallelAgent	sub_agents를 동시 실행 후 결과 병합	독립 작업 팬아웃(다중 소스 조사)
LoopAgent	조건/escalate까지 sub_agents 반복	반복 개선(작성→비평→재작성)

```
from google.adk.agents import SequentialAgent, LoopAgent

pipeline = SequentialAgent(
    name="content_pipeline",
    sub_agents=[researcher, writer, reviewer], # strict order
)
refine = LoopAgent(name="refine", sub_agents=[writer, critic],
    max_iterations=3) # or stop via escalate
```

워크플로 에이전트는 LLM 없이 흐름만 통제하므로 순서가 보장되고 비용이 들지 않는다. 루프 종료는 max_iterations 또는 하위 에이전트가 escalate 신호를 올리는 방식이다.

5 멀티 에이전트 — sub_agents·transfer·AgentTool

- sub_agents 계층 — 부모 LlmAgent가 하위 description을 보고 transfer(제어권 이양)로 동적 위임.
- AgentTool — 에이전트를 도구로 호출(제어권 유지, 결과만 수신). OpenAI SDK의 as_tool과 동형.
- 공유 state — 같은 세션의 에이전트들이 state로 데이터를 주고받음(output_key + 템플릿 주입).

```
from google.adk.tools.agent_tool import AgentTool

coordinator = Agent(
```

```

name="coordinator", model="gemini-2.5-flash",
instruction="Route billing to billing_agent; use summarizer as a tool.",
sub_agents=[billing_agent],          # transfer: hand over control
tools=[AgentTool(agent=summarizer)], # call as tool: keep control
)

```

6 Tools — 함수·내장·서드파티

파이썬 함수를 `tools=[fn]`에 넘기면 ADK가 시그니처·독스트링으로 자동 래핑한다(FunctionTool). 내장 도구로 `google_search`, 코드 실행기 등이 있고, OpenAPI 스펙·MCP 서버(MCPToolset)·LangChain/CrewAI 도구 래핑까지 지원해 도구 생태계가 넓다.

```

def get_exchange_rate(base: str, quote: str) -> dict:
    """Return the FX rate between two currencies."""
    return {"rate": 1350.2}

agent = Agent(name="fx", model="gemini-2.5-flash",
              tools=[get_exchange_rate])          # auto-wrapped as FunctionTool

```

7 Runner와 실행

Runner가 실행 엔진이다 — 에이전트·세션 서비스·아티팩트 서비스를 묶어 `run_async(user_id, session_id, new_message)`로 실행하고 Event 스트림을 돌려준다. 개발은 InMemoryRunner로 간단히, CLI로는 `adk run/adk web/adk api_server`를 쓴다.

```

from google.adk.runners import InMemoryRunner
from google.genai import types

runner = InMemoryRunner(agent=pipeline, app_name="demo")
session = await runner.session_service.create_session(
    app_name="demo", user_id="u1", state={"topic": "LangGraph"})

msg = types.Content(role="user", parts=[types.Part(text="go")])
async for event in runner.run_async(user_id="u1",
                                    session_id=session.id, new_message=msg):
    if event.is_final_response():
        print(event.content.parts[0].text)

```

8 Session·State·Memory

구분	역할	구현
Session	한 대화의 이벤트 이력 + state	InMemory / Database / VertexAI SessionService
State	세션 내 공유 dict(에이전트 간 데이터)	session.state, output_key, {key} 주입
Memory	세션을 넘는 장기 기억·검색	InMemory / Vertex AI Memory Bank

Session=단기(한 대화), **Memory=장기**(세션 횡단) 구분은 LangGraph의 체크포인트/Store 구분과 정확히 대응한다. 운영은 DB/Vertex AI 기반 서비스로 교체하면 코드 변경 없이 영속화된다.

9 Callbacks — 가드레일과 개입

에이전트 수명주기에 콜백을 걸어 검증·차단·변형한다: `before/after_model_callback`(LLM 호출 전후), `before/after_tool_callback`(도구 실행 전후), `before/after_agent_callback`. `before` 콜백이 값을 반환하면 **원래 호출을 건너뛰고 그 값을 사용** — 가드레일·캐시·정책 강제 지점이다.

```
def block_pii(callback_context, llm_request):
    if contains_pii(llm_request):
        return LlmResponse(content=types.Content(
            parts=[types.Part(text="Request blocked: PII detected.")]))
    return None # None -> proceed normally
```

```
agent = Agent(..., before_model_callback=block_pii)
```

10 개발 UI와 평가(adk web·adk eval)

- `adk web` — 브라우저 개발 UI: 대화 테스트, **이벤트·상태·트레이스 단계별 검사**, 그래프 시각화.
- `adk eval` — 평가셋(evalset)으로 에이전트 품질을 회귀 테스트. CI에 연결해 변경마다 검증.
- **관측** — Cloud Trace/OpenTelemetry 연동으로 운영 트레이싱.

11 배포 — Agent Engine·Cloud Run

- **Vertex AI Agent Engine** — 관리형 런타임: 세션·스케일·관측을 GCP가 운영(권장 경로).
- **Cloud Run / GKE / 컨테이너** — `adk api_server`로 API화해 어디든 배포.
- **Gemini Enterprise** 생태계와 연결되는 엔터프라이즈 경로 제공.

12 A2A — 에이전트 간 표준 프로토콜

A2A(Agent2Agent)는 구글이 주도해 Linux Foundation으로 이관한 **에이전트 간 통신 개방 표준**이다. 서로 다른 프레임워크·조직의 에이전트가 Agent Card로 능력을 공개하고 작업을 주고받는다. ADK는 에이전트를 A2A 서버로 노출하고 원격 A2A 에이전트를 하위 에이전트처럼 쓰는 것을 지원한다 — MCP가 "도구 연결" 표준이라면 A2A는 **에이전트 연결** 표준이다.

13 ADK vs LangGraph 비교

항목	Google ADK	LangGraph
추상화	에이전트 트리 + 워크플로 에이전트	상태 그래프 노드/엣지
핵심 모델	LlmAgent·Sequential/Parallel/Loop·Runner	State·Node·Edge + 체크포인트
제어 흐름	계층 위임 + 결정적 워크플로 중첩	그래프로 명시적 분기·순환
상태/영속	Session(state) + Memory 서비스	체크포인트 + Store
멀티 에이전트	sub_agents·transfer·AgentTool·A2A	서브그래프·Command 핸드오프
개발 경험	adk web UI·adk eval 내장	LangGraph Studio·LangSmith
배포	Agent Engine(관리형)·Cloud Run	LangGraph Platform·자체 호스팅
모델	Gemini 최적화(+LiteLLM 중립)	프로바이더 중립
대표 선택	GCP 스택·수명주기 통합·표준(A2A)	복잡 상태 머신·세밀한 체크포인트 제어

구도 요약: [GCP에 이미 서 있다면 ADK가 가장 마찰이 적고](#), 클라우드 중립·임의 토폴로지·체크포인트 수준의 정밀 제어가 우선이면 LangGraph가 유리하다. 둘 다 MCP를 지원하고, A2A로 상호 연동도 가능하다.

14 자주 하는 실수 모음

증상	원인	해결
위임이 엉뚱한 에이전트로 감	description 부실	라우팅 기준이 되는 명확한 description 작성
에이전트 간 데이터 단절	state 미활용	output_key 저장 + instruction {key} 주입
순서가 보장 안 됨	LLM 라우팅에만 의존	SequentialAgent로 결정적 구간 분리
루프가 안 끝남	종료 조건 부재	max_iterations 또는 escalate 설계
재시작 시 대화 소실	InMemory 서비스 사용	Database/VertexAi SessionService로 교체
google_search 오류	모델 호환 미확인	Gemini 계열 모델에서 사용
운영 품질 회귀	평가 부재	adk eval을 CI에 연결

15 API 빠른 레퍼런스

목적	코드
에이전트	<code>Agent(name, model, instruction, description, tools)</code>
상태 저장	<code>Agent(..., output_key="result")</code>
순차/병렬/반복	<code>SequentialAgent / ParallelAgent / LoopAgent(sub_agents=[...])</code>
계층 위임	<code>Agent(..., sub_agents=[child]) # LLM transfer</code>
에이전트를 도구로	<code>tools=[AgentTool(agent=other)]</code>
실행	<code>InMemoryRunner(agent, app_name) -> run_async(...)</code>
세션	<code>runner.session_service.create_session(app_name, user_id)</code>
가드레일	<code>before_model_callback / before_tool_callback</code>
개발 UI / 평가	<code>adk web / adk eval</code>
배포	<code>adk api_server / Vertex AI Agent Engine</code>

ADK는 빠르게 진화합니다. 본 문서는 2026-06 시점 기준이며, 정확한 시그니처는 google.github.io/adk-docs와 설치 버전 릴리스 노트로 최종 확인하세요.

Q

지식 점검 퀴즈 (객관식 20문항)

정답·해설은 다음 장에 있습니다. 보기 중 하나를 고르세요.

Q1. ADK 애플리케이션의 기본 구조는?

- A. 단일 거대 프롬프트
- B. 루트 에이전트를 정점으로 한 에이전트 트리
- C. 큐 기반 파이프라인
- D. SQL 스키마

Q2. 워크플로 에이전트 3종은?

- A. Map·Filter·Reduce
- B. Sequential·Parallel·Loop
- C. Start·Listen·Router
- D. Plan·Act·Verify

Q3. 워크플로 에이전트의 특징은?

- A. LLM이 순서를 정함
- B. LLM 없이 실행 흐름만 결정적으로 통제
- C. 항상 병렬
- D. 도구 전용

Q4. LLM 주도 동적 위임(transfer)의 판단 근거는?

- A. 에이전트 이름 길이
- B. 하위 에이전트의 description
- C. 알파벳 순서
- D. 토큰 수

Q5. 에이전트 최종 응답을 세션 state에 저장하는 설정은?

- A. save=True
- B. output_key
- C. memory=True
- D. @persist

Q6. instruction에서 세션 state 값을 주입하는 문법은?

- A. \$key
- B. {key}
- C.
- D. %key%

Q7. 제어권을 유지한 채 다른 에이전트를 호출하는 방법은?

- A. sub_agents
- B. AgentTool
- C. transfer
- D. escalate

Q8. 파이썬 함수를 도구로 쓰는 방법은?

- A. tools=[fn] — 자동 래핑(FunctionTool)
- B. 반드시 클래스 상속
- C. YAML 등록
- D. 불가능

Q9. 실행 엔진과 반환물은?

- A. Executor가 문자열 반환
- B. Runner가 Event 스트림 반환
- C. Compiler가 바이너리 반환
- D. Router가 dict 반환

Q10. Session과 Memory의 구분으로 옳은 것은?

- A. 동일 개념
- B. Session=한 대화(state 포함), Memory=세션을 넘는 장기 기억
- C. Memory가 단기
- D. Session은 배포 전용

Q11. LoopAgent의 종료 방법은?

- A. 외부 kill만
- B. max_iterations 또는 escalate 신호
- C. 토큰 소진
- D. 항상 1회

Q12. before_model_callback이 값을 반환하면?

- A. 에러 발생
- B. 원래 LLM 호출을 건너뛰고 그 값을 응답으로 사용
- C. 두 번 호출
- D. 무시됨

Q13. 브라우저에서 이벤트·상태를 단계별 검사하는 도구는?

- A. adk web
- B. adk lint
- C. adk shell
- D. adk logs

Q14. 에이전트 품질 회귀 테스트 도구는?

- A. adk eval
- B. adk test --unit
- C. pytest 전용
- D. 없음

Q15. 관리형 배포 경로는?

- A. Lambda
- B. Vertex AI Agent Engine
- C. Heroku
- D. S3

Q16. A2A 프로토콜의 역할은?

- A. 도구 연결 표준
- B. 프레임워크를 넘는 에이전트 간 통신 개방 표준
- C. 모델 가중치 교환
- D. 과금 API

Q17. MCP와 A2A의 관계로 가장 적절한 것은?

- A. 동일 표준
- B. MCP=도구 연결, A2A=에이전트 연결
- C. A2A가 MCP의 구버전
- D. 둘 다 구글 독점

Q18. ADK의 모델 지원에 대해 옳은 것은?

- A. Gemini 전용
- B. Gemini 최적화 + LiteLLM으로 타사 모델 가능
- C. OpenAI 전용
- D. 로컬 모델 불가

Q19. ADK vs LangGraph 비교로 가장 정확한 것은?

- A. ADK=그래프 저수준, LangGraph=트리 고수준
- B. ADK=에이전트 트리+수명주기 통합(GCP), LangGraph=명시적 그래프+체크포인트 정밀 제어
- C. 둘 다 GCP 전용
- D. 기능 동일

Q20. 에이전트 간 데이터 전달의 기본 패턴은?

- A. 전역 변수
- B. output_key로 state 저장 + instruction {key} 주입
- C. 파일 공유만
- D. 이메일

A

정답 및 해설

Q1 정답: B

ADK 앱은 루트 에이전트 정점의 트리다. sub_agents로 계층을 만들고 위임/워크플로를 중첩한다.

Q2 정답: B

SequentialAgent(순차), ParallelAgent(병렬), LoopAgent(반복).

Q3 정답: B

워크플로 에이전트는 LLM 호출 없이 흐름만 통제 — 결정적이고 비용이 없다.

Q4 정답: B

부모 LLM이 하위 에이전트의 description을 보고 transfer를 결정한다. 그래서 description 품질이 라우팅 품질.

Q5 정답: B

output_key를 주면 최종 응답 텍스트가 세션 state에 자동 저장된다.

Q6 정답: B

instruction의 {key}가 세션 state 값으로 치환된다.

Q7 정답: B

AgentTool은 에이전트를 도구로 호출(제어권 유지). sub_agents+transfer는 제어권 이양.

Q8 정답: A

함수를 tools에 넘기면 시그니처·독스트링 기반으로 FunctionTool로 자동 래핑된다.

Q9 정답: B

Runner.run_async가 Event 스트림을 반환하며 is_final_response()로 최종 응답을 식별한다.

Q10 정답: B

Session=한 대화의 이력+state(단기), Memory=세션 횡단 장기 기억. LangGraph의 체크포인트/Store와 대응.

Q11 정답: B

max_iterations 상한 또는 하위 에이전트의 escalate 신호로 종료한다.

Q12 정답: B

before 콜백이 값을 반환하면 원래 호출을 스킵하고 그 값을 사용 — 가드레일·캐시 구현 지점.

Q13 정답: A

adk web이 개발 UI로 대화 테스트와 이벤트·상태·트레이스 검사를 제공한다.

Q14 정답: A

adk eval이 평가셋 기반 회귀 테스트를 제공하며 CI에 연결할 수 있다.

Q15 정답: B

Vertex AI Agent Engine이 세션·스케일·관측을 제공하는 관리형 런타임이다.

Q16 정답: B

A2A는 프레임워크·조직을 넘는 에이전트 간 통신 개방 표준(Linux Foundation 이관).

Q17 정답: B

MCP는 도구(컨텍스트) 연결 표준, A2A는 에이전트 간 작업 교환 표준 — 상호 보완.

Q18 정답: B

Gemini에 최적화돼 있고 LiteLLM 연동으로 타사 모델도 쓸 수 있다.

Q19 정답: B

ADK는 트리+수명주기 통합(GCP 친화), LangGraph는 명시적 그래프와 체크포인트 수준의 정밀 제어.

Q20 정답: B

output_key 저장 + {key} 주입 조합이 에이전트 간 데이터 전달의 기본 통로다.